

国省统筹算法开发 技术规范 (第一版)

国省统筹算法研发小组
国家气象中心

1.术语与定义

数字智能天气预报业务算法：指平台后端用于气象预报相关数据处理、诊断、统计预测及服务等业务的一系列数学和计算方法，主要包括数据预处理、诊断分析、统计偏差订正、主客观融合等算法。本规范规定的业务算法，原则上应统一使用 python3.9 以上语言编写；如有其他语言(java/c 等)库依赖，需要撰写基于 python 调度其他库的接口。

国省统筹算法开发：通过顶层设计和资源整合，基于天擎集约化平台，实现国家级和省级数字智能天气预报算法的标准化、集约化改造和业务运行，遵循 “产权严格保护，团队合作开发” 的原则，打造国省算法改造-天擎测试-中试评估-应用验收的全链条统筹开发技术体系。建设国省两级数字智能天气预报体系的高效协同，提升高精度网格预报服务效能和响应速度。提升国省两级数字智能天气预报体系的高效协同、服务效能和响应速度。

算法插件(Plugins)：为完成数字智能天气预报算法中的某项独立核心功能，结合若干相关的基础函数/工具函数，构建的独立可复用的代码模块即为算法插件(Plugins)。算法插件(Plugins)需为符合统一接口规范的代码模块（通常以 Python 类形式实现）。算法插件的研发与标准化改造是国省统筹算法开发工作的核心。旨在将国家级及各省优质业务算法（如偏差订正、多源融合、降尺度等）中核心可复用部分，按照统一技术规范进行重构与封装，实现算法在国省集约

平台上的“一次研发、全国复用”与高效协同运行。国省算法插件(Plugins)测试验证后将按照功能放入 NIMM 算法框架对应目录。

NIMM 算法框架：NIMM (National Integration of Multi-Models Algorithm Framework)是指国家级多模式集成预报技术框架，也是国省统筹算法研发的核心算法插件(Plugins)资源池与标准底座。它不仅通过内置的标准化数据接口与成熟的基础算法插件(Plugins)（如偏差订正、多源融合等）为国省两级的算法开发与标准化改造提供强力支撑，更构建“共建共享、持续迭代”的反馈循环机制：即后续国省优质核心算法插件(Plugins)在通过统筹改造测试后，将反向纳入 NIMM 框架，实现算法资源的不断丰富与全国范围内的复用共享。

算法应用(CLI, 命令行界面)：是指基于算法插件 (Plugins) 的有序调用与组织，结合数据 IO、时空匹配及流程控制，构建的独立可直接运行的完整气象预报系统(CLI,command line interface, 命令行界面)。作为算法成果业务化的最终形态，CLI 应用封装了从数据输入、核心计算到产品输出的全流程链条，并针对天擎等集约化业务平台的运行环境进行了适配，能够满足低代码一键部署、多参数灵活配置及全流程监控等严格的业务运行要求，是实现国省两级算法高效落地与协同运行的关键载体。改造测试并通过研发小组代码审核后，算法应用(CLI)将上传至国省统筹算法库，后续按照版本控制管理进行算法应用(CLI)版本更新，并撰写相应更新说明。

国省统筹算法库：是指依托国家气象中心内网 GitLab 构建的，集约化管理国省两级优质算法成果的核心代码仓库。该库作为算法标

准化改造与业务化应用之间的关键关口，负责接收并托管经过测试验证的算法应用（CLI），通过执行严格的代码审核机制与精细化的版本管理，确保入库算法的合规性、稳定性与可复用性，从而实现全国范围内算法资源的统一发布与高效共享。

国省统筹算法研发平台：是指依托天擎（CMADAAS）云资源构建的，集成了 NIMM 算法框架基础镜像、高性能计算与存储资源、统一预处理大数据环境的**标准化研发支撑体系**。该平台旨在解决国省两级研发环境割裂、数据标准不一的难题，为算法的标准化开发、存量算法改造及仿真测试提供一站式、集约化的底层环境支持，保障算法应用（CLI）成果能够高效地在国省两级业务系统中无缝移植与复用。

meteva 程序库：国家气象中心预报技术研发室检验科负责研发的程序库，旨在为从数值模式、客观方法、精细化网格预报到预报产品的应用的整个气象产品制作流程中的每个环节进行快速高效的检验，促进跨流程跨部门的检验信息共享，为推进研究型业务和改进预报质量提供技术支撑。

meteva_base 基础数据包：meteva_base 为 meteva 程序库中的数据、读写、处理计算及简单绘图等工具的汇总，增加部分统计后处理相关接口和属性，**是国省统筹算法插件(Plugins)的底层统一数据底座**，可以支持大部分数字智能气象算法的基础读写转换计算工作，是保障算法插件(Plugins)复用性的关键支撑。

2.国省统筹算法技术规范

数字智能天气预报业务算法是指服务于国省两级数字气象网格预报、客观预报及衍生服务等业务的后端预报系统。为落实“国省统筹”研发工作方案，本规范依托国家级多模式集成预报技术框架（NIMM）和天擎（CMADAAS）云资源，旨在确立统一的技术标准，打破国省技术壁垒，实现算法在国省两级业务体系中的集约化运行与高效协同。

本规范主要目的是加强国省统筹算法成果的规范化管理及深度复用，明确算法插件(Plugins)与算法应用(CLI)的分级管理策略。其中，具备通用性、可复用性的核心功能模块应封装为核心算法插件(Plugins)，经验证后纳入NIMM 算法框架，作为基础能力供全国共享；而由多种算法插件(Plugins)构建而成的面向具体业务场景、包含完整调度流程的算法应用(CLI)，则需在天擎测试评估后统一上传至国省统筹算法库进行全生命周期管理，以满足天擎等集约化平台的自动化调度与多参数控制要求。

国省统筹算法开发遵循“产权严格保护，团队合作开发”的原则。鼓励国省两级组建联合研发团队，开展技术攻关。在实现代码集约化管理的同时，充分尊重和保障开发者的知识产权与成果归属，构建“共建、共享、共用”的良性研发通过。代码合规性修改、单元测试及文档编写等工作由开发团队协同完成，确保算法的可移植性与稳定性。

成立国省统筹算法管理团队，依托内网 GitLab 代码仓库，**建立严格的算法库管理、代码审核及版本管理机制**。在系统业务化评审、工程项目验收或国省统筹算法遴选前，开发者需按照本规范上传算法成果。管理团队将对入库 NIMM 的算法插件与入库算法库的 **算法应用 CLI** 应用分别进行技术审核，并出具意见。只有通过审核并完成版本归档的算法，方可正式进入国省集约化平台进行中试应用。

2.1 数据格式

2.1.1 数据来源与结果存储

业务算法的数据来源为天擎，输出结果在天擎中保存。

2.1.2 数据格式

业务算法使用的数据格式包括数字基座中保存的各类数据格式和一体化平台定义的业务算法标准数据格式(详见附件 2)。如果需要增加新的数据格式，可按照数字基座规范增加。

(1) 站点数据结构

站点数据格式(STDA)，为固定格式的 `pandas.DataFrame` 数据，包括 6 列维度信息(高度、时间、预报时效、站点、经度、纬度)以及若干数据列,同时包括 6 个基础数据属性(数据单位、要素名、预报时效单位、时间时区、高度类型及要素起止时间)。

(2) 网格数据结构

网格数据(GRD),为固定格式的 `xarray.DataArray` 数据，包括 6 维维度信息(成员、高度、时间、预报时效、经度、纬度)以及若干数

据维度,同时包括 6 个基础数据属性(数据单位、要素名、预报时效单位、时间时区、高度类型及要素起止时间)。

(3) 线条数据结构

线条数据(LINE),为固定格式的 `geopandas.GeoDataFrame` 数据,包括站点、线条及多边形面类型,同时包括 6 个基础数据属性(数据单位、要素名、预报时效单位、时间时区、高度类型及要素起止时间)。

为支撑一体化平台业务算法标准格式数据,国家气象中心自研统一数据支撑库(`meteva_base`, <https://www.showdoc.com.cn/metevabase>),提供上述数据初始化、读写、转换等工具函数,数据格式及服务接口具体规定见附件 1(算法手册)。

2.2 算法模块化

算法设计遵循模块化原则,通过将复杂业务逻辑分解为不同层级的功能单元,实现代码的高内聚、低耦合。本规范推荐采用“基础函数 -> 算法插件(Plugins) -> 算法应用(CLI)”的三级架构,以确保算法在国省统筹背景下的可复用性、可维护性及高效集约化运行。

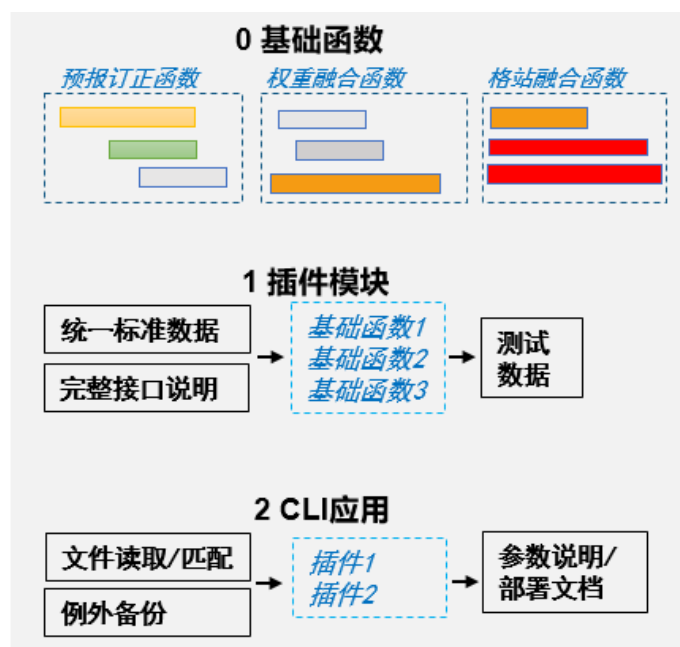


图 1 算法模块化设计示意图

2.2.1 基础函数与工具函数 (Functions)

基础函数是指为完成特定数学计算或单一数据处理任务而编写的代码单元，通常以函数形式封装。若基础函数在多个模块中被复用，应提升为工具函数进行统一管理。

开发规范：

1. 数据类型：
 - 输入输出应优先采用标准基础数据类型（如 Numpy 数组、Python 集合等），减少对特定复杂对象的依赖，提升通用性。
2. 计算效率要求：
 - 核心计算逻辑必须采用向量化运算（Vectorization）；
 - 严禁在处理大规模网格数据（Grid Data）时使用原生 Python for 循环，应利用 Numpy、Scipy 或 Xarray 等高性能库实现数组运算。
3. 命名与封装：
 - 模块内部私有函数应采用下划线开头（如 `_func()`）命名，最小化功能暴露；
 - 工具函数应具备必要的关键注释，明确输入输出定义。

2.2.2 算法插件 (Plugins)

算法插件是数字智能预报业务中的核心逻辑单元，指结合若干基

础函数构建的、独立且可复用的代码模块。插件是本规范管理的重点对象。

设计原则：

1. 核心复用：

所有具备业务独立性及复用价值的功能模块（如气温地形降尺度、位涡诊断、短中期降水订正等）均应封装为算法插件。

2. 颗粒度对齐：

功能相似的算法应整合成统一插件，通过参数配置区分不同场景（例如：500hPa 与 850hPa 的气旋识别应整合为同一算法，通过高度层参数控制）。

开发规范：

1. 代码组织：

- 插件必须以 **Python 类 (Class)** 形式提供。
- 必须通过 `__init__()` 方法进行参数初始化设置。
- 必须通过 `process()` 函数作为主入口调用核心功能。
- 代码中需包含符合文档规范的详细注释（`Docstring`）。

2. 输入输出与 I/O 解耦：

- **严禁文件 I/O**：插件内部**严禁**出现任何文件路径的读写操作（如 `open()`, `read_csv()` 等）。
- **内存对象处理**：插件的输入输出必须为内存对象（如 `pandas.DataFrame` 或 `xarray.DataArray`）。文件读写必须剥离至 CLI 层处理，以满足“云原生”架构及“流水线”内存流转的部署要求。
- **环境无关性**：插件运行仅依赖于输入参数，不得包含隐藏的上下文环境依赖（如硬编码的路径、全局变量等）。

3. 数据标准：

- 输入输出的站点/网格/线条数据对象，必须符合 **2.1 节定义的统一标准数据格式**。
- 标准数据格式之外的属性（如数据单位、变量名、阈值等），必须以**显式参数**的形式在接口中定义，禁止隐式依赖。

4. 完整性与健壮性：

- 插件内部应包含完整的数据预处理流程，包括数据有效性检查、时间维度的匹配与对齐、缺测值处理等。

5. 参数规范：

- 输入输出参数命名应具有高可读性，符合气象业务领域惯例（推荐命名参考附录 1）。

2.2.3 算法应用 (CLI)

算法应用 (CLI) 是指通过有序调用一个或多个算法插件，结合数据 I/O、时空匹配及流程控制逻辑，构建的独立可执行气象预报系统。

CLI 面向业务部署，需满足天擎等集约化平台的运行要求。

开发规范：

1. 代码组织：

- CLI 应编写在单独的脚本文件中，负责数据的读取（Load）、插件调度（Invoke）及结果写出（Dump）。
- 执行主程序推荐命名为 process() 或遵循 if __name__ == "__main__": 标准入口模式。

2. 运行控制：

- **一键式运行**：必须支持通过命令行参数（如 argparse）配置输入输出路径、起报时间、预报时效等关键信息。
- **低代码部署**：避免将复杂的气象数据对象作为直接参数，参数设计应简洁明了，便于调度系统配置。

3. 全流程运维：

- CLI 必须包含完整的**异常处理机制**，能够捕获并记录错误信息。
- 必须按照规范输出**标准日志**（Logging）及**进程退出码**（Exit Code），以便调度系统监控运行状态。

2.2.4 算法运行

将功能独立、复用性高、内部“高内聚”强相关的接口功能进行组织，打包成算法插件(Plugins)。常见算法如位涡分析诊断、气温地形降尺度、降水偏差订正、多模式融合等。算法插件可以独立运行，也可以由一个或若干个算法插件组合后形成算法应用 CLI 后部署运行。

2.3 算法插件(Plugins)

2.3.1 算法插件(Plugins)功能分类

算法插件(Plugins)指用于气象数据处理、诊断和统计预测的一系列数学和计算方法，它们的主要目标是利用各种数据来源进行气象要素数字天气预报，是数字智能预报系统的核心部分。常用算法插件

(Plugins)包括数据预处理、诊断分析、统计偏差订正、主客观融合等功能，本规范中可分为 10 类预报算法和 2 类辅助算法。

预报算法包括：

1. 时间/空间降尺度算法(space/time downscale)
 - 提供将时空粗分辨率的数据降尺度到更精细的分辨率功能。
2. 观测处理算法(obs_adustment)
 - 提供多源观测数据(站点、探空、雷达、卫星等)质控及其他相关处理的算法。
3. 诊断算法(diagnostic)
 - 用于从原始数据中诊断新的气象要素变量。如天气现象、降雪过程雪水比(cobb)等。
4. 临近预报算法(nowcast)
 - 提供基于实况数据的临近(0-3h)预报算法。
5. 偏差订正算法(single_calibration)
 - 提供单一模式的校准模型，消除系统偏差，使其更接近真实观测值。
6. 多源融合和多时效拼接(integrate blending)
 - 将多个模型/多时效的输出进行自适应融合，以提高预报的准确性。
7. 灾害及极端预报算法(extreme)
 - 包括极端事件(极端强降水、再现期)及客观灾害事件(高温、寒潮、地质水文等)预报算法。
8. 概率预报算法(probability)
 - 基于集合模式或多源模式数据，进行概率预报计算、邻域提取及可靠性订正等算法
9. 主客观融合及协调算法(consistense)
 - 提供主观预报/灾害预报预警与客观预报融合，以及多气象要素间、站点格点预报间相互协调的算法。
10. 检验算法(verification)
 - 与多源预报相关的产品检验算法，包括站点、格点及目标检验等。

其他算法包括：

1. 数据读写预处理(I0)
 - 对多源数据读写的支撑，包含一些函数，用于将多源输入数据和输出结果与统一网格预报数据格式的转换。相关算法融入 meteva_base 库
2. 辅助算法(ancillaries)
 - 包含一些辅助函数或数据，用于支持其他模块的运行，如地形掩码、地理信息计算、数据复制或裁剪等。

2.3.2 分类命名

为统一管理业务算法插件(Plugins),按照算法种类,给算法类规定统一的前缀名,具体对应如下:

算法种类	算法前缀	NIMM 框架算法路径
空间降尺度算法	<i>SDS_</i>	<i>/00space_downscale</i>
时间降尺度算法	<i>TDS_</i>	<i>/00time_downscale</i>
观测处理算法	<i>OBS_</i>	<i>/01obs_adustment</i>
诊断算法	<i>DIA_</i>	<i>/02diagnostic</i>
临近算法	<i>NOW_</i>	<i>/03nowcast</i>
偏差订正算法	<i>CAL_</i>	<i>/04single_calibration</i>
多源融合	<i>MUL_</i>	<i>/05multi_integrate</i>
多时效拼接	<i>BLD_</i>	<i>/05blending</i>
灾害及极端预报算法	<i>EXT_</i>	<i>/06extreme</i>
概率预报算法	<i>PRO_</i>	<i>/07probability</i>
主客观融合及协调算法	<i>CONS_</i>	<i>/08consistense</i>
检验算法	<i>VER_</i>	<i>/09verification</i>
辅助算法	<i>ANC_</i>	<i>/ancillaries</i>
数据读写预处理	<i>IO_</i>	<i>/basic_data</i>

表 1 算法(算法插件, PLUGINS)统一命名前缀

如:

基于多源融合的短中期降水预报: MUL_MaitPrecipitation24H()、

基于地形的气温降尺度: SDS_TemperDownScaleByTopo()。

如有其他算法无法归纳到当前种类中,需要增加种类时,可以申请算法管理团队审查并统一分配算法前缀名。

2.3.3 测试脚本和算法指南

算法插件(Plugins)交付时必须包含完整的**测试数据**及**测试执行脚本**。**测试数据**应采用符合国省统筹标准格式(如标准站点/网格数据)的轻量化代表性样本,涵盖常规气象场景及边界情况。开发者需提供**算法插件(Plugins)notebook 测试脚本**(如 run_test.ipynb),确保审核人员在部署环境后能运行算法插件(Plugins),验证其可用

性与稳定性；

算法插件需提供详细的接口注释和代码规范。插件的核心类（Class）及主要方法（如 `__init__` 和 `process`）中，必须有详细的注释说明；除代码注释外，鼓励开发者提供独立的算法说明指南。具体见 2.4、2.5 节。

2.4 算法应用(CLI)

国省统筹及其他工程项目开发的算法成果，在中试和验收前原则上需以算法应用（CLI）的形式，上传至国家气象中心内网 GitLab 国省统筹算法仓库。仓库管理遵循“组建团队合作开发，产品归原开发者”的原则开源共享，并由业务算法管理团队对仓库的读写、修改权限进行统一控制。

一个标准的算法应用（CLI）应包含以下核心组件：

- **核心算法插件：**经过测试和优化的算法插件（Plugins）及其对应的 API 接口说明；
- **执行入口：**基于插件构建的、集成了数据 IO 与流程控制的系统入口（Entry Points），确保可直接在天擎等集约化环境中执行；
- **配套文档：**算法插件（Plugins）清单、算法应用系统简介(详见 2.6)。
- **目录结构：**算法应用的具体文件内容及目录结构需严格符合 2.4.2 章节规范。

算法应用上传至 GitLab 后，业务算法管理团队将负责审查其代码规范、命名规则及文档合规性（详见 2.8）。对于不合规的提交，团队将提出修改建议；只有通过代码审核的算法应用，方可正式纳入国省统筹核心算法库并进行版本归档。

2.4.1 算法应用(CLI)相关文档

上传算法包时，需同时提供算法包内所有算法插件的功能简介，内容如下：

算法插件	功能(尽量详细)	路径
IO_ReadNcFromCmadaas	从天擎读取 NC 文件，返回标准网格数据	/src/read/readgrid.py
Cal_SoilTemperMax24h	基于模式及站点实况，生成订正后的土壤日最高温	/src/cal/calsoiltemper.py
.....		

同时，建议提交一份算法应用(CLI)简介，参考业务化评审中系统简介部分，重点涵盖输入输出、技术流程与评估效果三大核心要素。明确界定算法的输入数据及输出产品的具体规格（时空分辨率、数据单位及格式）。声明其预期的内存和 CPU 核心峰值，用于申请天擎仿真部署资源；以流程图展示全链路逻辑；并重点提供基于基准预报数据集（Benchmark）的客观检验评估结果（如 TS 评分提升率、RMSE 降低幅度或运行效率优化比），直观展示算法的业务性能优势与集约化运行价值，作为算法通过审核及业务准入的核心依据。

2.4.2 算法应用(CLI)命名规范

为支持算法应用(CLI)的统一管理及复用，按算法主要功能将算法包分为以下 6 类。上传前，需按照类型进行统一的命名调整。具体如下：

- **数据 IO 及预处理：**包括对多源数据读写、解码、转换计算的支撑；多源观测数据(卫星、雷达等)的质控、拼接等预处理等算法。
- **诊断：**包括多种诊断量计算及相应可视化等算法。
- **数字智能预报：**包括多基于模式的统计后处理订正及气象要素确定性/概率客观订正算法(单模式)。
- **多源融合及一致调整：**包括多源、多时效融合，站点网格融合，主客观融合，多要素一致性等算法。

- **灾害及极端事件：**包括极端事件(极端强降水、再现期)及灾害事件(高温、寒潮、地质水文等)的客观预报预警等算法。
- **检验：**与多源预报相关的产品检验算法。

算法包种类	算法包前缀
数据 IO 及预处理	<i>MetIO_</i>
诊断	<i>MetDig_</i>
统计后处理	<i>MetCal_</i>
多源融合及一致调整	<i>MetBlend_</i>
灾害及极端	<i>MetExt_</i>
检验	<i>MetEva_</i>

表 2 算法包统一命名前缀

如有其他算法包无法归纳到当前种类中，则需要增加算法包种类时，可以申请算法管理团队审查并统一分配算法包前缀名。

2.4.3 算法应用 (CLI) 通用的文件/目录结构

为了提高代码可读性和维护性，算法应用 (CLI) 开发需遵循以下文件和目录结构：

- **(1) src/**
目录存放源代码,包括基础函数和算法插件(Plugins)等，小写字母和下划线命名源代码文件。
- **(2) test/**
目录存放单元测试文件，以 `test_` 开头的同名代码。
- **(3) docs/**
目录存放算法相关文档，用描述性命名相关描述文件 (如 `sorting_algorithm.pdf` 等),包括算法应用 CLI 中算法清单及集成测试检验效果等，确保文档内容清晰。
- **(4) resource/**
目录存放算法所需的资源文件(地形、站点表等)。
- **(5) utils/**
目录存放公共工具函数。
- **(6) cli/**
目录存放基于相关算法插件(Plugins)构建的算法应用 CLI 应用，不同功能(见第七章)的 CLI 采用不同前缀进行区分。
- **(7) nbs/**
目录存放 notebook 算法示例代码，主要是算法插件(Plugins)测试和示例脚本。
- **(8) env/**
目录存放算法包运行时所依赖环境包，推荐使用 `pip (requirements.txt)` 或 `conda(environment.yml)` 的自动部署工具。部署表中应标明执行不同任务

时必装和选装的环境。

- **(9) license**
算法包对应的 License 项目授权。
- **(10) setup.py**
pip 安装算法包的对应安装配置列表。

2.4.4 算法应用 (CLI) 版本管理

算法版本使用语义版本控制 (Semantic Versioning)，如

‘1.0.0’。版本号格式为主版本号.次版本号.修订号，其中：

- **主版本号：**有重大变更，可能不兼容的更新。
- **次版本号：**新增功能，兼容性良好的更新。
- **修订号：**小的修复和改进，完全兼容的更新。

算法应用 (CLI) 版本更新，应附带详细的更新记录，包括算法更改，效果评估，贡献人等信息。

2.5 代码编写规范

2.5.1 开发环境

业务算法编写语言及环境为 python(>3.9)，确保所有开发者在该环境下进行开发和升级，避免版本冲突或错误。

2.5.2 算法插件 (Plugins) 及函数命名规范

基础函数、算法插件应使用有意义且简介的名称，便于理解记忆和管理。函数 function 及类 class 等命名，应严格依照 python 编码规范。具体规定如下：

- **1. 基础函数：**函数名一律小写，如有多单词用下划线隔开；私有函数在函数名前加一个下划线_，如 `def _private_func()`。
- **2. 算法插件及其他类：**类名应使用驼峰结构(PascalCase)，不使用_连词，如 `class AnimalFarm()`。
- **3. 变量：**变量名尽量小写，常量采用全大写，如有多个单词，用下划线隔开。

2.5.3 代码编写

采用 python 作为算法库基础编码语言，遵循 PEP8 代码风格规范，使用 flake8 或 black 等工具进行代码格式化检查。

确保代码格式化良好；变量和函数命名具有可读性；添加适当的注释，解释算法的工作原理和关键步骤；使用清晰易懂的代码逻辑；避免过于复杂的表达和嵌套语句；使用异常处理机制；定期进行代码审查，确保代码风格的合规和统一。

2.5.4 算法开发中其他注意事项

算法开发过程中应注意的事项还包括：

- 编写基础函数时，目标功能应当单一；减少基础函数中逻辑分支，使更容易通过代码测试，并且更容易维护。
- 针对需求定义基本算法插件类，功能尽量独立且单一；算法插件中尽量调用基础函数，最好只调用插件中其他方法。
- 本应该是函数的东西就要写成函数，而不是类（即基本功能尽量以函数形式，如果合适在插件 `plugins` 层级再使用类）。
- 利用注释等功能，加强代码可读性。比如在做复杂数组或多维数据集操作时，需要考虑对非专业开发人员的可读性。
- 确保函数和类经过了单元准确性测试，并在函数和方法接口上使用类型注释(`type annotations`)。
- 模块化代码编写，代码较多(如总计>5000 行)时，可以考虑制作为模块。当功能需要不止一个函数来完成时，可以类和模块的方式进行模块化处理。
- 注意使用简单数据结构，如推荐使用 32 位浮点数(非 64 位)。
- 强化私有 `private` 概念，算法应采用最小知识原则，对外暴露东西越少越好。如 `class` 类中，对外暴露的方法要尽量减少；模块 `module` 中，对外提供的接口也尽量减少。
- 开发过程中，如有多模块、多个类中都使用到的函数，应做成工具函数，并存放在对象都能访问的共享位置。
- 严禁在算法核心计算逻辑中使用 Python 原生循环处理大规模格点数据，必须使用 Numpy/Xarray 等库进行向量化运算

2.5.5 算法接口注释

算法插件的代码注释需严格遵循 Python 文档字符串 (Docstring) 规范 (推荐 Google 或 NumPy 风格)。在插件的核心类 (Class) 及主要方法 (如 `__init__` 和 `process`) 中, 必须详细有详细注释。注释建议包括以下部分:

- **Args:** 必需参数。
- **Keyword Args:** 关键字参数。
- **Raises:** 引发的异常。
- **Returns:** 函数或方法返回的变量。
- **References:** 可用文档的链接。
- **Warns:** 引发的警告

2.5.6 算法类型注释

python 默认无定义变量及类型, 在大型算法的搭建或测试中容易引起错误。特别强调, 输入输出参数及返回值必须使用类型注释 (Type Hinting), 明确标识数据类型 (如 `xarray.DataArray`, `pandas.DataFrame` 等), 以增强代码的可读性和 IDE 的静态检查能力, 降低第三方调用时的理解成本。

2.6 文档及测试规范

2.6.1 算法(Plugins/CLI)指南

除代码注释外, 鼓励开发者提供独立的算法说明指南 (通常为 Markdown 或 PDF 格式)。指南内容应包括算法原理 (数学公式或物理机制)、输入输出详细定义 (包括单位、时空分辨率要求)、关键参数配置说明 (默认值及调整建议) 以及运行示例。完善的算法插件测试脚本和指南是通过代码审核并纳入 NIMM 框架及国省统筹算法库

的前置必备条件。开发者应在算法开发过程中，编写对应的说明文档，帮助其他开发者理解及应用。算法指南推荐包括以下内容：

- **1. 算法名称和目的：**明确指出算法的名称和主要目的。
- **2. 输入和输出：**包括输入/输出参数的类型、格式和范围。如多个输入/输出，分别列出具体的含义。
- **3. 算法步骤：**可以使用文字、伪代码或流程图等方式来说明算法步骤的操作和顺序。
- **4. 时间复杂度和空间复杂度：**分析算法的时间复杂度和空间复杂度，以评估算法的效率和资源消耗。
- **5. 算法示例：**示例应包括输入数据、预期输出和实际运行结果。
- **6. 边界条件和异常处理：**说明算法在边界条件下的处理方式。指出算法对于异常情况的处理方法，如输入错误、越界等。
- **7. 可选参数和定制选项：**明确说明可选参数的含义、用法和默认值。并说明如何根据不同的需求来调整算法的行为。
- **8. 测试用例：**测试用例包括输入与输出，应覆盖常见的算法试用情况。
- **9. 参考文献和引用：**在文档中提供参考文献或算法的列表。
- **10. 统计检验评估结果：**研发阶段算法的实际检验评估效果，说明其应用场景、优点和不足。

2.6.2 算法单元测试

算法内需包含单元测试和集成测试，确保算法功能的正确性和稳定性。

单元测试需基于 `pytest` 等标准框架编写，覆盖插件的初始化、核心计算及异常处理流程，重点验证输入输出数据维度的正确性及计算结果的逻辑合理性；确保在运行良好情况下生成正确输出；确保根据需要引发异常或警告。新算法上线前，需提供与原算法（如有）或基准算法在单元测试运行耗时和内存占用上的对比说明。

2.6.3 算法插件快速集成测试

算法插件(Plugins)应提供对应的测试示例数据及一键执行的算法集成测试脚本(notebook)，可以“一句话”快速对某算法插件进行

测试，确保审核人员在部署环境后能通过简单指令快速跑通全流程，验证插件的可用性与稳定性。

2.6.4 算法异常及日志

算法应包括较完整的日志，以便跟踪算法的执行过程和性能，并提供监控信息。对应日志记录要求需满足“天镜”监控平台需求，包括运行状态、读写记录、产品生成记录、异常记录等。

日志必须采用标准格式（建议 JSON），并使用 logging 模块的标准级别。算法 CLI 进程退出时，必须返回符合 POSIX 标准的 Exit Code，以便调度系统识别任务状态（例如：0 成功，1 数据缺失（非致命），2 计算错误（致命）等）。

2.7 算法审核及发布规范

2.7.1 算法应用(CLI)统一上传及管理

中心成立统一的业务算法管理团队，依托国家气象中心内网 GitLab 代码托管平台，搭建“国省统筹算法仓库”并负责统一管理。所有经过改造及测试的算法应用（CLI），均须通过 Git 版本控制工具上传至该仓库，实行全生命周期的代码管理。**算法包必须基于指定的 NIMM 基础镜像环境进行开发和测试，禁止算法私自升级基础底座库（meteva_base, numpy, xarray 等），如需新特性需向管理团队申请升级基础镜像。**

（1）仓库管理与权限

算法应用(CLI)的源码提交、版本更新及分支管理统一在 GitLab 国省统筹算法仓库中进行。算法开发者需申请相应仓库权限,遵循“一人一号”原则。算法管理团队负责仓库架构维护、权限分配及代码合规性审查。

（2）开源协议与许可

算法应用 (CLI) 首次提交时,根目录下应包含相应的 LICENSE 开源协议许可文件 (参考: <https://segmentfault.com/a/1190000041599305>)。推荐在气象中心算法库中使用 GPL3 (开源) 或 BSD2/3 (修改后闭源, 授权推广) 等协议,明确知识产权归属与使用授权。

（3）产权保护

国省统筹算法开发成果归原开发者所有,在**算法清单和代码头部进行统一开发者标注**。设置引用规范,如算法 B 复用了算法 A “公用类” 插件,需要在算法 B 中保留原开发者标注。

（4）打包与发布

算法应用 (CLI) 每次版本更新后,应包含标准的 `setup.py` 或 `pyproject.toml` 文件,支持打包为 `pip` 在线安装包 (Wheel/Egg)。这旨在确保算法应用能够以标准 Python 包的形式,在天擎等集约化业务环境中通过 `pip install` 命令快速部署和更新。算法每次更新需提供版本日志,记录每个版本更新内容

（5）环境依赖规范

为确保算法应用在国省两级环境中的复用性与无缝衔接,业务算

法管理团队将制定统一的基础依赖环境标准（即 NIMM 基础镜像环境）。算法应用必须对主要基础计算库（如 numpy, scipy, pandas, sklearn, xarray, cartopy 等）的版本范围进行严格限制，确保算法能够在统一版本的基础依赖环境中通过测试并正常运行。

2.7.2 算法应用(CLI)审核

国省统筹算法管理团队负责算法可见权限管理(全部可见，部分可见，授权)，区分为公用类+私人类(权限控制)，具体由使用的 license 开源协议或开发者权限申请中给定。

管理团队**负责算法包的审核**，包括代码及命名、文档的合规性，如不合规提出修改建议。开发者应继续修改算法包直至通过审核。

2.7.3 算法代码审核规范

算法库采用基于 git 的拉取请求方式(pull request, PR)进行不同成员间的代码审核及更新。main 分支为稳定发布版，开发者应在 feature 分支开发。统一使用 flake8 等代码审查工具，方便团队协作和代码管理。

推荐代码更新时只进行较小的 PR(小于几百行)，大的 PR 难以审查，可能导致错误；一个 PR 对应一个功能点或一个 Bug 修复，严禁将无关代码修改混在一个 PR 中提交。

代码审核时，每小时不要超过 500 行代码审查速度，一次审查不超过 400 行代码或连续一小时以上。

2.8 国省统筹算法天擎研发平台

数字智能天气预报算法，应统一基于上述原则进行 python 工程化改造及测试部署，并融入天擎集约化环境。

研发平台依托天擎（CMADAAS）云资源构建，集成了 NIMM 基础框架、统一数据环境及仿真测试工具，为国省两级算法的标准化开发与集约化改造提供底层支撑。平台使用需遵循以下五项规范。

2.8.1 研发平台天擎资源申请和分配

遵循“按需申请、统筹分配、动态调整”的原则。算法开发团队（包括国家级及省级团队）需根据研发项目的计算复杂度与数据吞吐量，向管理团队提交计算及存储资源申请。管理团队审核通过后，统一分配仿真容器资源，确保研发与业务运行环境一致性。

2.8.2 基础 NIMM 镜像加载

研发平台提供统一的 NIMM 基础算法镜像，并实时更新。开发者在构建算法开发环境时，必须加载该官方基础镜像。镜像中已预置了标准化的 Python 运行环境（Python 3.9+）及受控版本的核心依赖库（如 NIMM、meteva_base、numpy、xarray 等）。除特殊说明外，开发者原则上不得擅自修改基础镜像中的核心库版本，以保障算法应用（CLI）在国省两级环境间的无缝移植。

2.8.3 统一数据预处理库（环境）

研发平台直接挂载天擎“国省统筹统一预处理数据专题库(NIMMData)”，包含经过标准化处理的、长序列（2-3 年）及实时的多模式预报（ECMWF，CMA-GFS 等）与多源实况（网格/站点）数据。国省统筹算法开发过程中，应直接利用标准数据接口读取 NIMMData，禁止私自搭建重复的原始数据处理流程，以减少算力与存储冗余，提升研发效率。

2.8.4 数据算法注册及仿真评估

算法应用（CLI）在完成初步开发后，需在研发平台(天擎)进行数据和算法在线注册。并基于仿真天擎调用历史 NIMMData 数据集进行批量回算。仿真平台运行指标(包含资源占用、稳定性及 EI/DI 信息等)将作为算法应用业务迁移的初审依据。

2.8.5 业务迁移及中试运行

通过仿真评估的算法应用，将业务迁移至天擎加工流水线并实时部署。中试期间，研发平台对结果进行实时检验，与原始数值模式进行客观对比分析。平台将动态展示算法性能，包括准确率（如 TS 评分、RMSE 提升率）、稳定性（偏差波动情况）及时效性等。测试期(一个月以上)后上述指标将作为中试评审验收的核心材料。

附录 1 算法参数

参数 (中文)	参数 (英文)
站点数据	<i>sta/station</i>
站点数据(预报)	<i>sta_fcst/station_fcst</i>
站点数据(实况)	<i>sta_obs/station_obs</i>
网格数据	<i>grd/grid</i>
网格数据(预报)	<i>grd_fcst/grid_fcst</i>
网格数据(实况)	<i>grd_obs/grid_obs</i>
站点信息数据	<i>station_info</i>
网格信息数据	<i>grid_info</i>
数据值	<i>values/data</i>
数据单位	<i>units</i>
经度	<i>lon/lons/longitude</i>
纬度	<i>lat/lats/latitude</i>
经度间隔	<i>dx/dlon</i>
纬度间隔	<i>dy/dlat</i>
起始经度	<i>lon_start/lon_s/slon</i>
起始经度	<i>lat_end/lon_e/elon</i>
结束纬度	<i>lat_start/lat_s/slat</i>
结束纬度	<i>lat_end/lat_e/elat</i>
经度数	<i>nx/nlon</i>
纬度数	<i>ny/nlat</i>
预报成员	<i>member</i>
预报时间	<i>time</i>
预报时效	<i>dtime</i>
预报层次	<i>level</i>
维度	<i>dim/dimension</i>
阈值	<i>thr/threshold</i>
百分位数	<i>percentiles</i>
掩膜	<i>mask</i>
海陆掩膜	<i>land_sea_mask</i>
范围 (时间)	<i>bound/ time_bounds</i>
范围(空间)	<i>range/ space_range</i>
时间步长	<i>timestep/timesteps</i>
坐标	<i>coordinate/coord</i>
坐标系统	<i>coord_system</i>
要素	<i>element</i>
文件名 (通配)	<i>fmt/file_fmt</i>
文件路径	<i>file_path/path</i>
插值方法	<i>interp_method</i>
变换方法	<i>transform_method</i>
输入	<i>input</i>

输出

| *output*

附件 2：国省统筹算法开发技术手册